

SOFTWARE REQUIREMENTS SPECIFICATION

ContextBridge

Persistent, agent-independent memory for developers

Version 0.1 | Phase 1 Requirements Draft | 03 October 2026

Status: Provisional requirements; architecture and implementation are intentionally not defined in this document.

1. Introduction

1.1 Purpose

This SRS defines the Phase 1 requirements for ContextBridge, a local-first persistent memory layer intended to preserve useful developer and project context across AI agents and sessions. It establishes the problem, scope, functional requirements, quality attributes, constraints, and acceptance criteria before architecture design.

1.2 Problem Statement

AI coding agents can maintain context within a session and, in some products, across sessions tied to the same agent. Context is less reliable when a developer changes agent, model, IDE, or session. ContextBridge addresses this gap by providing an external, persistent memory that is independent of any single agent or model.

1.3 Goals

- Preserve important developer and project context across sessions and heterogeneous AI agents.
- Avoid uncontrolled memory growth by selectively admitting and consolidating useful information.
- Detect and manage contradictory or superseded memories rather than blindly overwriting them.
- Retrieve a small, relevant set of memories with temporal and scope awareness.
- Start as a single-developer, local-first system while keeping the data model ready for team and server deployment.

2. Scope

In Scope (V1)	Out of Scope (V1)
Developer-level and project-level memory	Team collaboration workflows
Cross-session and cross-agent persistence	Cross-device synchronization
Model-agnostic memory interface	General-purpose chatbot memory
Selective memory admission and asynchronous consolidation	Full codebase archival
Conflict detection and supersession/history	Raw tool-log storage
Temporal and scope-aware retrieval	Complex multi-agent orchestration
Local-first deployment	Large-scale multi-tenant production service
Manual memory management capability in the data model	Full management UI in initial core implementation

3. Users and Use Cases

3.1 Primary User

Individual software developers using multiple AI coding agents, IDEs, or models. The initial implementation targets one developer; the design should remain extensible to teams and multiple users.

3.2 Core Use Cases

ID	Use Case	Expected Outcome
UC-01	Continue work in a new session	Relevant prior project context is available without repeating it.
UC-02	Switch AI agent/model	The new agent can access the same authorized developer/project memory.
UC-03	Switch project	Project-specific memory remains isolated from other projects.
UC-04	Change a project decision	The new state supersedes the old state while preserving history.
UC-05	Recall a prior solution or failed approach	A concise, relevant memory and its provenance can be retrieved.
UC-06	Inspect or remove stored memory	The developer can manage persistent memories explicitly.

4. Functional Requirements

ID	Requirement	Description
CB-FR-01	Persistent Storage	The system shall persist approved memories beyond the originating session.
CB-FR-02	Cross-Agent Access	The system shall expose memory through an interface that is independent of a specific AI model or agent.
CB-FR-03	Memory Scopes	The system shall support developer-global and project-specific memory scopes.
CB-FR-04	Scope Isolation	Project-scoped retrieval shall not return memories from unrelated projects unless explicitly requested and authorized.
CB-FR-05	Selective Admission	The system shall not store every interaction as long-term memory; it shall identify candidate information worth retaining.
CB-FR-06	LLM-Based Semantic Processing	Memory extraction, semantic admission, consolidation, and conflict interpretation shall be LLM-assisted and model-configurable.

ID	Requirement	Description
CB-FR-07	Asynchronous Processing	Non-critical memory processing shall occur outside the normal response path where feasible.
CB-FR-08	Conflict Detection	The system shall check new candidate memories against relevant existing memories for contradictions.
CB-FR-09	Supersession	When a new memory invalidates an earlier one, the system shall represent the newer memory as current/superseding while retaining historical provenance.
CB-FR-10	Temporal Validity	Memories that change over time shall support time-related metadata sufficient to distinguish current from historical information.
CB-FR-11	Provenance	Each persistent memory shall retain source metadata such as originating agent/client, session, timestamp, and applicable project/scope.
CB-FR-12	Retrieval	The system shall retrieve relevant memories using a selective retrieval strategy rather than returning the complete memory store.
CB-FR-13	Adaptive Retrieval Depth	The system shall support a small default retrieval set with the ability to expand when query complexity or uncertainty requires more context.
CB-FR-14	Hybrid Retrieval	Retrieval shall be designed to support semantic similarity together with lexical/symbolic or metadata constraints where useful.
CB-FR-15	Durable Outcomes	The system shall prefer storing durable outcomes of tool activity instead of raw execution logs.
CB-FR-16	Code References	Where memory refers to code, the system should store references or identifiers rather than duplicating the codebase as long-term memory.
CB-FR-17	Secret Protection	The system shall prevent or redact common secrets and credentials before persistent storage.
CB-FR-18	Manual Management	The system shall provide data-level support for inspect, edit, delete, restore, and obsolete/retired operations; a full UI may follow later.
CB-FR-19	Pluggable LLM	The memory-processing LLM provider shall be replaceable, including support for future local-model deployment.
CB-FR-20	Explicit Memory	The system should support an explicit developer action such as “remember this” for information that must be persisted immediately.

5. Memory Content Requirements

V1 should prioritize memory that is durable and useful for future software-development work:

- Project facts: technology choices, architecture facts, dependencies, environment constraints.
- Developer preferences and conventions: coding style, preferred technologies, workflow conventions.
- Decisions and constraints: accepted/rejected approaches, reasons, non-negotiable project constraints.
- Task and project state: current status, unresolved issues, dependencies, outstanding work.
- Lessons and solutions: important bugs, root causes, fixes, failed approaches, and reusable outcomes.

Assistant-generated suggestions shall not be treated as authoritative memory merely because an agent produced them. They should be stored as durable knowledge only when supported, adopted, or otherwise validated by the memory policy.

6. Memory Lifecycle Requirements

Stage	Requirement
Capture	Collect interaction/session information without immediately treating every turn as long-term memory.
Candidate Extraction	Convert useful information into compact, self-contained memory candidates.
Admission	Evaluate utility, reliability, redundancy, and contextual applicability before persistence.
Conflict Check	Compare candidate memories with related stored memories before committing changes.
Consolidation	Merge related or repetitive information when doing so preserves useful detail and reduces redundancy.
Versioning	Represent changed facts as superseding or historical states rather than silently deleting the past.
Indexing	Maintain searchable representations and scope/time metadata.
Retrieval	Select only memories relevant to the current task, respecting scope and temporal validity.
Management	Allow explicit inspection, correction, retirement, deletion, and restoration of memories.

7. Retrieval Requirements

- Retrieval shall be query-driven and scope-aware.
- Temporal constraints shall be considered for queries involving changing facts or historical events.
- The system shall prefer a small high-value context over indiscriminate retrieval of many memories.
- Retrieval shall be able to combine semantic, lexical, and structured metadata signals.
- The initial target is a small default retrieval set (approximately 3 memories), with adaptive expansion subject to evaluation.

7.1 Retrieval Safety Principle

Retrieved memories are context for the agent, not unquestionable truth. Conflicting, stale, low-confidence, or weakly supported memories should be distinguishable through their state, provenance, or confidence metadata.

8. Non-Functional Requirements

ID	Quality Attribute	Requirement
CB-NFR-01	Latency	Ordinary local retrieval should target approximately <500 ms p95; this is an engineering target to be validated experimentally.
CB-NFR-02	Responsiveness	Heavy extraction/consolidation shall not block normal agent interaction where asynchronous execution is possible.
CB-NFR-03	Privacy	Persistent memory shall be local-first in V1 and shall minimize storage of secrets and unnecessarily sensitive content.
CB-NFR-04	Isolation	Memory access shall be restricted by developer/project scope.
CB-NFR-05	Model Independence	The system shall not require a specific foundation model for memory storage or retrieval.
CB-NFR-06	Extensibility	The design shall accommodate future server deployment, multiple developers, and multiple clients without changing core memory semantics.
CB-NFR-07	Auditability	Memory state changes should be traceable through provenance and history.
CB-NFR-08	Efficiency	The system should minimize unnecessary LLM calls and avoid repeated processing of unchanged memory regions.
CB-NFR-09	Maintainability	Memory-processing, storage, retrieval, and integration concerns should remain separable so components can be evaluated independently.
CB-NFR-10	Reliability	The system should fail safely when memory is unavailable, ambiguous, conflicting, or malformed; normal agent operation should not be permanently blocked.

9. Constraints and Assumptions

- V1 targets one developer and a local machine.
- The project is initially a research/engineering prototype, not a production multi-tenant service.
- Memory processing may use a cloud LLM initially; the provider must remain replaceable.
- A local model is a future option, not a V1 requirement.
- The initial client set will be limited to feasible AI coding clients; exact clients are to be finalized during implementation planning.
- The system will use external, explicit memory rather than modifying model parameters.
- Full team collaboration, cross-device sync, and a full management interface are future scope.

10. Security and Privacy Requirements

- Memory writes shall include a pre-storage secret/sensitive-data screening step.
- Common credentials such as API keys, passwords, access tokens, and private keys shall not be persistently stored.
- Project isolation shall be enforced at retrieval as well as storage.
- The system should preserve enough provenance to determine where a memory originated without exposing unnecessary raw conversation history.
- Deletion requirements shall support eventual removal from active indexes and storage.

11. Research Direction and Candidate Novelty

The current research direction is not claimed as a final novelty contribution. Existing literature already covers persistent memory, conflict-aware ingestion, temporal state, and efficient consolidation in different combinations. ContextBridge will investigate whether a distinct contribution can be established around the developer-specific, cross-agent setting.

Candidate research focus:

- Agent-independent developer/project memory shared across heterogeneous coding agents.
- Conflict handling when memories originate from different agents, sessions, or clients.
- Efficient asynchronous consolidation that reduces redundant memory while preserving current state, historical context, and provenance.

This direction must be validated against the full literature and experimental results before being stated as the project contribution.

12. Initial Acceptance Criteria

- A fact written in one session can be retrieved in a later session without re-entering it.
- A second supported agent/client can retrieve the same authorized memory without depending on the first agent's model state.
- Project-specific memory does not leak into an unrelated project retrieval.
- A changed fact results in distinguishable current and historical states rather than an ambiguous duplicate.
- Low-value interactions are not persistently stored as long-term memory in normal operation.
- Memory processing can run asynchronously without blocking the primary interaction path.
- Retrieval returns a small relevant set and can apply temporal/scope constraints.
- Stored memories contain provenance sufficient to identify their originating source/session.
- Common secrets are rejected or redacted before persistence.
- A developer can inspect and remove a stored memory through the implemented management mechanism.

13. Research Basis

The requirements above were derived from the research corpus reviewed for ContextBridge, especially the following findings:

Source	Requirement Implication
HaluMem	Memory extraction and updating can omit important information, introduce hallucinated information, and create update errors; memory quality should therefore be evaluated at extraction, update, and QA levels.
MemoryAgentBench	Robust memory requires accurate retrieval, test-time learning, long-range understanding, and selective forgetting rather than recall alone.
LongMemEval	Multi-key indexing and time-aware retrieval improve retrieval of long-term conversational information; retrieval quality is affected by representation and time constraints.
SimpleMem	Semantic compression, consolidation, and adaptive retrieval can reduce redundancy and context cost; retrieval depth should not simply be increased without evidence.
LightMem	Memory construction and heavier updates can be separated from the real-time interaction path; offline/async consolidation can improve efficiency.
MOSAIC	Save-time conflict detection and local re-evaluation address error compounding, showing that conflict-aware memory is an established research direction.
StateMem / TOKI	State tracking, supersession, temporal validity, and audit/history are important when stored facts evolve over time.
A-MEM	Structured, dynamically linked memory can evolve as new information arrives, but this may introduce additional LLM processing cost.
Lost in the Middle	More context is not automatically better; relevant information can be underused when buried in long retrieved contexts.
Memory in the Age of AI Agents	External/token-level memory is transparent, addressable, transferable, and suitable for model-agnostic persistent systems.

14. Requirements Status

Area	Status
Problem definition	Defined for Phase 1
Primary user	Defined: individual developer
Developer/project scope	Defined provisionally
Memory content	Defined provisionally
Memory lifecycle	Defined provisionally
Retrieval strategy	Defined provisionally
Privacy/security	Defined provisionally
Deployment	Local-first V1
Technology stack	Not fixed in this SRS
Supported clients	Not fixed in this SRS
Final research novelty	Open; requires literature validation and experiments
System architecture	Intentionally deferred to the next phase

Note: "Provisional" requirements are intentionally allowed to change after detailed literature analysis, feasibility testing, and evaluation design. No architecture choice is implied by this SRS.